Section: 14. Functions

Lecture: 155. Variable Length Positional Arguments

Section 1: Lecture Summary

This lecture introduces **variable length positional arguments** in Python functions, explaining their use, syntax, and behavior with clear examples. Starting from familiar positional arguments, the instructor explains how built-in functions like `print()` accept any number of positional arguments, motivating the need for user-defined functions that can do the same. The core syntax involves placing an asterisk `` before a parameter name (commonly `args`), which allows a function to accept **any number of positional arguments**, collected as a tuple inside the function.

The lecture demonstrates this concept with a function `fun(args)` that prints all arguments passed, showing that `args` is a tuple. It then explores iterating over `args` with a `for` loop and filtering based on argument type (e.g., printing only integers). The lecture also clarifies how to combine fixed positional parameters before or after the variable length argument, emphasizing:

- Arguments before `args` must be **positional-only** (cannot be passed as keywords).

- Arguments after `args` must be **keyword-only** (must be passed with keywords).

Finally, it covers the behavior of passing lists to variable length arguments, highlighting that passing a list directly treats the whole list as one argument inside `args`, whereas unpacking the list with `` passes each item as separate positional arguments.

Throughout, various code examples test these concepts and errors, providing a comprehensive understanding of how variable length positional arguments function in Python.

Section 2: Key Concepts and Explanations

- **Variable Length Positional Arguments (`args`):**

  - The syntax `def fun(args):` allows a function to accept any number of positional arguments.

  - Inside the function, `args` is a **tuple** containing all passed positional arguments.

  - Example: `fun(1, 2, 'hello')` → `args` becomes `(1, 2, 'hello')`

- Supports arguments of any type simultaneously (int, float, string, etc.).

- **Iterating Over Variable Arguments:**

- Since `args` is a tuple, you can iterate using a `for` loop:

```
    for x in args:
        print(x)
```

- Conditional checks can filter arguments based on type or other properties:

```
    for x in args:
        if type(x) == int:
            print(x)
```

- **Mixing Fixed Parameters with Variable Length:**

- You can define fixed positional parameters **before** `args`:

```
    def fun(a, b, *args):
        print(a, b, args)
```

- Here, `a` and `b` must be passed as **positional arguments**.

- Remaining positional arguments get collected into the tuple `args`.

- You cannot pass fixed parameters before `args` as keywords; doing so raises a syntax error:

- Correct: `fun(10, 20, 30)`

- Incorrect: `fun(a=10, b=20, 30)` → Error: positional argument follows keyword argument.

- **Keyword-only Arguments After `args`:**

- If you declare parameters **after** `args`, they must be **keyword-only**:

```
    def fun(*args, a, b):
        print(a, b, args)
```

- These arguments cannot be passed positionally and must use keywords when calling:

```
    fun(10, 20, 30, a=40, b=50)
```

- Omitting keywords raises `TypeError` indicating missing required keyword-only arguments.

- **Passing Lists to Variable Length Arguments:**

- Passing a list directly to `args` makes the entire list a single element in the tuple:

```
    L1 = [10, 20, 30]
    fun(L1)  # args = ([10, 20, 30],) – tuple with one list element
```

- To unpack the list elements as separate positional arguments, use the unpacking operator ``:

```
    fun(*L1)  # args = (10, 20, 30)
```

- **Summary of Parameter Passing Rules with Variable Arguments:**

Parameter Position	Passing Style Allowed	Note
Before `*args`	Positional only	Cannot pass as keyword arguments
`*args` itself	Collects all extra positional	Collected as a tuple internally
After `*args`	Keyword only	Must be passed as keywords, else TypeError

---

Section 3: Example Code and Use Cases

Example 1: Simple variable length arguments

```
def fun(*args):
    print(args)

fun(10)
fun(10, 12.5)
fun(10, 12.5, 'hello')
```

Prints tuple of passed arguments each time.

Example 2: Iterating through *args and printing one by one

```
def fun(*args):
    for x in args:
        print(x)

fun(5, 1.25, 'hello', 15)
```

Prints each argument on a new line.

Example 3: Filtering *args to print only integers

```
def fun(*args):
    for x in args:
```

```
        if type(x) == int:
            print(x)

fun(5, 1.25, 'hello', 15)
```

Prints only integer arguments (5 and 15).

Example 4: Fixed positional parameters before *args

```
def fun(a, b, *args):
    print(a, b, args)

fun(10, 20, 30)         # a=10, b=20, args=(30,)
fun(10, 20, 30, 40, 50) # a=10, b=20, args=(30, 40, 50)
```

Shows fixed parameters fixed and remaining in tuple.

Example 5: Keyword-only parameters after *args

```
def fun(*args, a, b):
    print(a, b, args)

fun(10, 20, 30, a=40, b=50)
```

`a` and `b` must be passed via keywords.

Example 6: Passing a list directly to *args

```
def fun(*args):
    print(args)
    print(len(args))

L1 = [10, 20, 30]
fun(L1)       # args = ([10, 20, 30],)
fun(*L1)      # args = (10, 20, 30)
```

Demonstrates that passing list directly passes one argument (the list), passing with unpacks elements.

Section 4: Key Takeaways

- Use `args` in function definitions to allow **variable number of positional arguments**.

- The variable length positional arguments are packed into a **tuple** inside the function.

- You can iterate over `args` and apply filtering or processing on the arguments.

- When mixing fixed parameters with `args`:

- Parameters before `args` must be **positional-only** (cannot be passed as keywords).

- Parameters after `args` must be **keyword-only** (must be passed with explicit keywords).

- Passing a list directly to a `args` parameter treats the whole list as a single argument.

- Use unpacking `list` to spread list elements as individual positional arguments.

- These features make Python functions highly flexible compared to many other languages and enable dynamic argument passing patterns.